

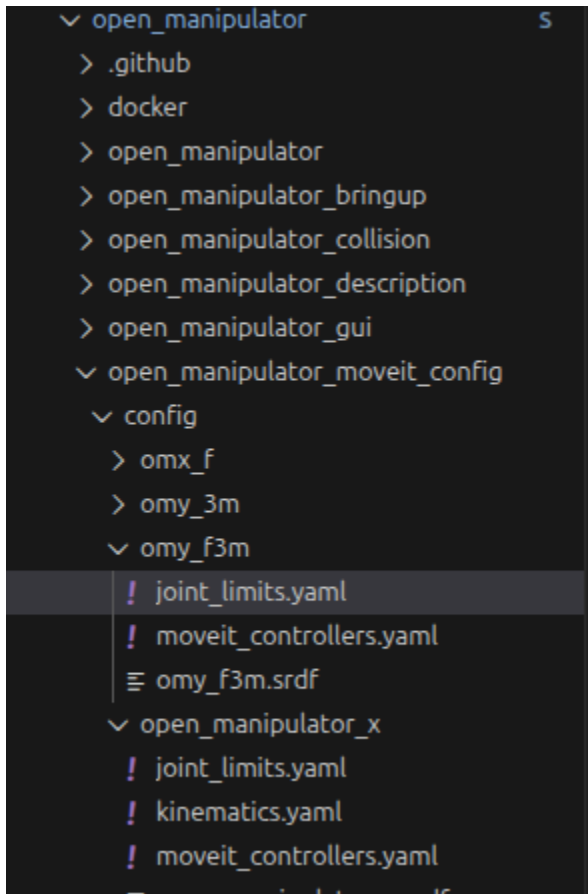
# Following MoveIt Tutorials

## Getting Started - video walkthrough

- Following all step on this page:
  - Clone the main branch: `git clone -b main https://github.com/moveit/moveit2_tutorials`
  - Needed to colon build with the key word `--executor sequential` or my laptop freezes.
  - Successfully followed all the steps.

## MoveIt Quickstart in RViz

- Default velocity and accelerator are limited to 10%. These can be changed in the `moveit_config` in `joints_limit.yaml`.



- Saving the rviz config works but save as freezes rviz. I will ignore this issue for now.

## Your First C++ MoveIt Project

- The node that commands the robot to go to an arbitrary task space pose does not need to import the robot description. However, there needs to be a node that is publishing a topic with the robot description. For this demo, it's [demo.launch.py](#) that is publishing the robot description.
- target pose is set (via `setPoseTarget`). The starting pose is implicitly the position published by the joint state publisher, which could be changed using the `MoveGroupInterface::setStartState*` family of functions.
- Successfully completed this section.

## Visualizing In RViz

- Successfully completed this section.

## **Planning Around Objects**

- Successfully completed this section.

## **Motion Planning Python API**

- Successfully commanded the robotic arm in the tutorial with python.

## **Planning Scene Monitor**

RobotState

- a snapshot of a robot. Contains RobotModel and a set of joint values.

CurrentStateMonitor (CSM)

- ROS wrapper for RobotState.
- Subs to a JointState topic that provides up to date joint sensor values.
- Uses TF2 buffer that uses TF2 TransformListener.

PlanningScene

- Snapshot of the world: includes RobotState and any collision objects.
- Can be used for collision checking.

## Commanding OMY in taskspace with moveit

- Successfully commanded omy in gazebo.
- **A config file:**
  - set planning scene mintor options such as topics for robot description joint space, etc.
  - Set planning pipeline parameters.
- **A moveit node:** special kinda node
  - Create an instance of a moveitpy node.
  - Set it's planning component i.e. arm or gripper.
  - This node has methods in that such as current state, start state, goal state etc.
  - It does trajectory generation so must set start and goal state (pose) to move the robot.
  - For taskspace pose, need to be [PoseStamped\(\)](#) ros msg.
    - Need to understand quaternians for real bruh.
  - This pose need a reference point. For us it will be 'world' or "link\_0" or was it "link0" (gotta check).
  - For the goal state, the pose (a PoseStamped() msg) and the link("end\_effector\_link" for us) that needs to go to the pose needs to be set.
- **A launch file:** moveitpy nodes needs a bunch of configuration parameters such as robot description, planning pipeline, etc.
  - Used the omy's moveit launch file as a reference to pass the correct paramters/files to moveitpy.

03/2/2026

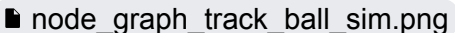
## Why does OMPL fail and take so long?

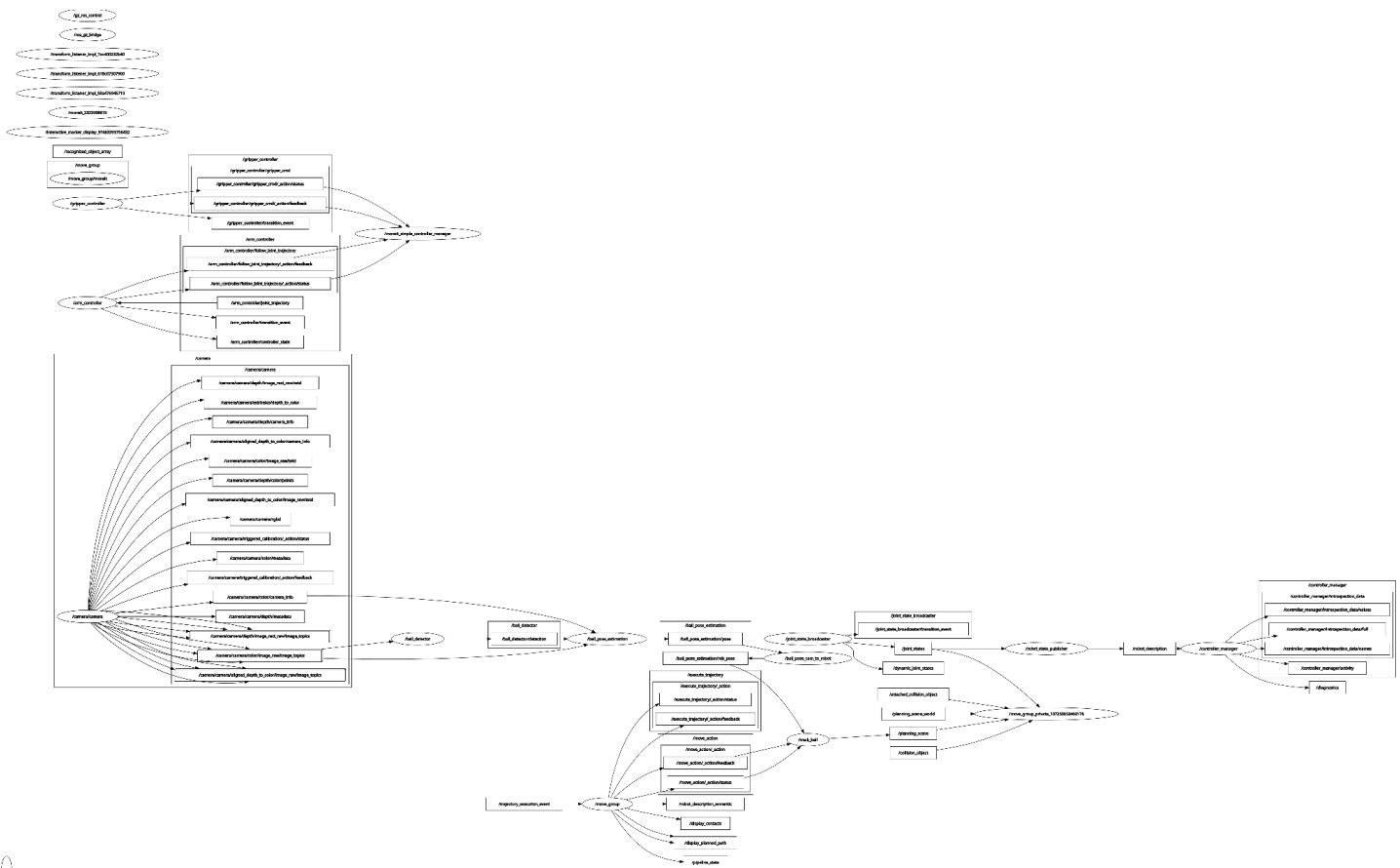
I have a hunch that OMPL is more capable and faster than we think it is. It is the default planner in Moveit. There's gotta be a reason for it. I don't think we are using it properly.

- This time, I followed the example in the pkg open\_manipulator\_playground.

I created the following:

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pkg         | my_motion_planner                                                                                                                                                                                                                                                                                                                                                                                                                          |
| nodes       | Track_ball.py, <ul style="list-style-type: none"><li>- associated launch: track_ball.<a href="#">launch.py</a>,</li><li>- No custom config needed</li></ul>                                                                                                                                                                                                                                                                                |
| Description | This uses ompl, move_group. The robot pose is updated every 2 secs.<br><br>WORKS IN SIM:<br>IN SIM, If the ball doesn't move, the robot still moves to the same ball pose. I think since ompl is nondeterministic, it's computing different traj to the same pose every time. We can probably figure out how to only command robot if the ball has moved a certain distance so the robot doesn't take a long traj to where it already was. |
| How to run  | Run gazebo and this node's launch file with use_sim:=true                                                                                                                                                                                                                                                                                                                                                                                  |

When running this with real camera, node graph is: 



## Use MoveIt servo:

- [Realtime Servo — Moveit Documentation](#)
- [https://github.com/moveit/moveit2/blob/main/moveit\\_ros/moveit\\_servo/launch/demo\\_ros\\_api.launch.py](https://github.com/moveit/moveit2/blob/main/moveit_ros/moveit_servo/launch/demo_ros_api.launch.py)
- [https://github.com/moveit/moveit2/blob/3758a0fa/moveit\\_ros/moveit\\_servo/src/collision\\_monitor.cpp](https://github.com/moveit/moveit2/blob/3758a0fa/moveit_ros/moveit_servo/src/collision_monitor.cpp)
- [https://github.com/moveit/moveit2/blob/3758a0fa/moveit\\_ros/moveit\\_servo/src/servo\\_node.cpp](https://github.com/moveit/moveit2/blob/3758a0fa/moveit_ros/moveit_servo/src/servo_node.cpp)
- 
- First I got the panda example working.
- Then I used keyboard to control the panda.  

```
ros2 run moveit_servo servo_keyboard_input
```
- I saw that pressing keys published msgs to the following two topics:  

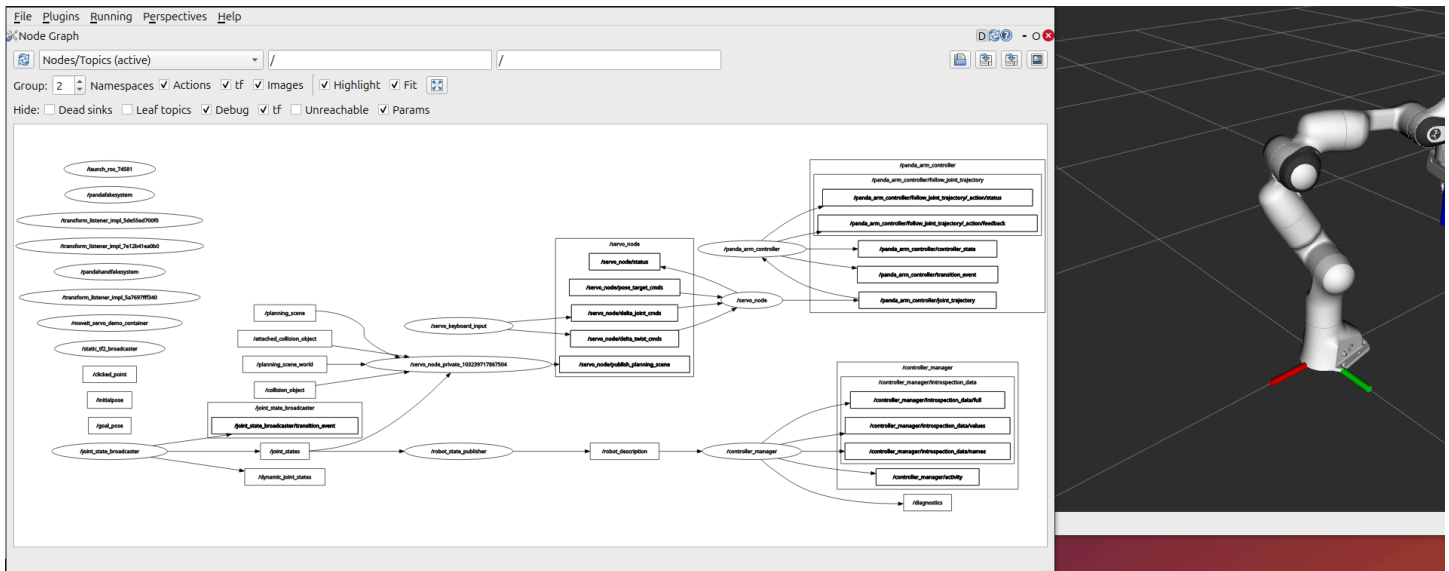
```
/servo_node/delta_twist_cmds
```

```
/servo_node/delta_joint_cmds
```
- So to fake these kinda commands, i created the following file. In the future, a tracking ball node can publish like this node.  

```
fake_pub_delta_twist.py
```
- There's no node that I wrote for this. I just wrote the config file and launch file.  

```
config/my_servo_params.yaml,
```

```
launch/my_servo.launch.py
```
- /arm\_controller in node graph must be connected to the robot specific topics. See img.



```
ashik@ashik-Legion-S7-15ACH6:~/throw_and_catch/throw_and_catch/throw_and_catch_ws$ ros2 param describe /servo_node move
it_servo.lower_singularity_threshold
ros2 param describe /servo_node moveit_servo.hard_stop_singularity_threshold
ros2 param describe /servo_node moveit_servo.leaving_singularity_threshold_multiplier
ros2 param describe /servo_node moveit_servo.joint_limit_margins
Parameter name: moveit_servo.lower_singularity_threshold
Type: double
Description: Start decelerating when the condition number hits this (close to singularity)
Constraints:
  Min value: 0.0
  Max value: 1.7976931348623157e+308
Parameter name: moveit_servo.hard_stop_singularity_threshold
Type: double
Description: Stop when the condition number hits this
Constraints:
  Min value: 0.0
  Max value: 1.7976931348623157e+308
Parameter name: moveit_servo.leaving_singularity_threshold_multiplier
Type: double
Description: When 'lower_singularity_threshold' is triggered, but we are moving away from singularity, move this many
times faster than if we were moving further into singularity
Constraints:
  Min value: 0.0
  Max value: 1.7976931348623157e+308
Parameter name: moveit_servo.joint_limit_margins
Type: double array
Description: Added as a buffer to joint variable position bounds [in that joint variable's respective units]. Can be
of size 1, which applies the margin to all joints, or the same size as the number of degrees of freedom of the active j
oint group. If moving quickly, make these values larger.
Constraints:
  Min value: 0.0
ashik@ashik-Legion-S7-15ACH6:~/throw_and_catch/throw_and_catch/throw_and_catch_ws$
```

I added “ `allow_nonzero_velocity_at_trajectory_end: true` ” # ashik added this. Allows the controller to end a trajectory with nonzero velocity, which maybe important for our use case of tracking a moving target. If false, the controller will try to decelerate to zero velocity at the end of each trajectory, which can cause issues when trying to track a moving target.” to `/home/ashik/throw_and_catch/throw_and_catch/throw_and_catch_ws/src/open_manipulator/open_manipulator_bringup/config/omy_f3m/hardware_controller_manager.yaml`.

**send joint angles to robot (CLI common):**

```
ros2 topic pub --once /arm_controller/joint_trajectory trajectory_msgs/msg/JointTrajectory "{
  joint_names: ['joint1','joint2','joint3','joint4','joint5','joint6'],
  points: [{
    positions: [-1.5708, 0.0, 1.5708, -1.5708, 1.50708, 0.0],
    time_from_start: {sec: 3}
  }]
}" # goes to home pose
```

```
ros2 service call /servo_node/switch_command_type moveit_msgs/srv/ServoCommandType
"{command_type: 1}"
```

This kinda works, but we need to understand moveit servo params better.

TODO:

Read more documentation.

What was that website that explains git repo?

Trying the full param list.

## Relearn the concept of Moveit:

[https://github.com/moveit/moveit2\\_tutorials/tree/main](https://github.com/moveit/moveit2_tutorials/tree/main)

<https://moveit.picknik.ai/main/api/html/index.html>

- Moveit\_ros is the ROS2 wrapper that has move\_group node and PlanningSceneMonitor.

Fucntions/methods to master:

- MoveItConfigsBuilder
- PlanningPipeline

<https://moveit.picknik.ai/main/doc/concepts/concepts.html>

```
# home:
ros2 topic pub --once /omy_move1_controller/move1 robotis_interfaces/msg/MoveL "{
  pose: {
    pose: {
      position: {x: -0.13, y: -0.40, z: 0.54},
      orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}
    }
  },
  time_from_start: {sec: 3, nanosec: 0}
}"

# close to catching:
ros2 topic pub --once /omy_move1_controller/move1 robotis_interfaces/msg/MoveL "{
  pose: {
    pose: {
      position: {x: 0.30, y: -0.45, z: 0.6},
      orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}
    }
  },
  time_from_start: {sec: 3, nanosec: 0}
}"
```